

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
public static void BindStock(this DropDownList ddl, string schoolid)
{
    string sql =string.Format(@"select id [key],stockname [value] from
categorystock where schoolid='{0}' ", schoolid);
    ddl.InitBySql(sql);
}
public static void BindCategoryStock(this DropDownList ddl, string CategoryId)
{
    string sql = string.Format(@"select stockid [key],stockname + '(' +
cast(ISNULL(num,0) as varchar) +isnull(unit,'') +')' [value]
                                from CategoryNum where CategoryID='{0}' ",
CategoryId);
    ddl.InitBySql(" ");
}
public static void BindCategoryManager(this DropDownList ddl, string schoolid)
{
    string sql = string.Format(@"select c.teacherid as [key],c.name as [value]
from(
select a.teacherid ,a.name from teacherinfo a, CategoryKindManager b where
a.teacherid=b.employeecode and b.schoolid='{0}' and a.schoolid='{0}'
)c group by c.teacherid ,c.name ", schoolid);
    ddl.InitBySql(sql);
}
public static void BindCategoryApplyManager(this DropDownList ddl, string
schoolid,string step)
{
    string sqldoman = string.Format(@"select b.domanid from flowbase b where
b.schoolid='{0}'
                                and b.step={1} and b.flowkind='', schoolid, step);
    string
domanid=SessionHolder.CreateTemplate().ExecuteScalarString(sqldoman);
    if (domanid.IsNull())
    {
        return;
    }
    string sql = string.Format(@"select a.teacherid as [key],a.name as [value]
from teacherinfo a
                                where a.teacherid in ({1}) and a.schoolid='{0}'
order by a.name ", schoolid, domanid);
    ddl.InitBySql(sql);
}
public static void BindCategoryDept(this DropDownList ddl, string schoolid)
{

```

```

        string sql = string.Format(@"select a.deptId [key],a.deptName [value] from(
                                select deptId,deptName,'1' type,'' classCode,''
classindex from dept
                                where schoolid='{0}' and deptName<>''
                                union all
                                select classid deptId,classname deptName,'2'
type ,SUBSTRING(classCode,1,5) classCode,
                                serialnumberingrade classindex
                                from ClassInfo where schoolid='{0}')) a
                                order by a.type,a.classCode,a.classindex",
schoolid);
        ddl.InitBySql(sql);
        ddl.Items.Insert(0,new System.Web.UI.WebControls.ListItem("", ""));
    }
    public static void BindApplyDept(this DropDownList ddl, string schoolid)
    {
        string sql = string.Format(@"select a.deptId [key],a.deptName [value] from(
                                select deptId,deptName from dept
                                where schoolid='{0}' and deptName<>'' ) a",
schoolid);
        ddl.InitBySql(sql);
        ddl.Items.Insert(0, new System.Web.UI.WebControls.ListItem("", ""));
    }
    public static void BindCategoryTeacher(this DropDownList ddl, string schoolid)
    {
        string sql = string.Format(@"select a.teacherId [key],a.name [value]
                                from TeacherInfo a where SchoolId='{0}'
                                order by a.name", schoolid);
        ddl.InitBySql(sql);
        ddl.Items.Insert(0, new System.Web.UI.WebControls.ListItem("", ""));
    }
    public static void BindCatagoryUseDept(this Ext.Net.Store store)
    {
        string sql = string.Format(@"select usedeptid as [key],usedeptname as [value]
from categoryuse
                                where categoryid in (select id from categoryinfo where
schoolid='{0}'))
                                group by usedeptid,usedeptname order by usedeptname
                                ", SecurityHolder.CurrentUser.SchoolId);
        DataTable dt = ObjectDataAdapter.ExecuteDataset(sql);
        store.DataSource = dt;
        store.DataBind("");
    }
    public static void BindCatagoryUseUser(this Ext.Net.Store store)
    {
        string sql = string.Format(@"select userid as [key],username as [value] from
categoryuse
                                where categoryid in (select id from categoryinfo where
schoolid='{0}'))
                                group by userid,username order by username

```

```

        ", SecurityHolder.CurrentUser.SchoolId);
    DataTable dt = ObjectDataAdapter.ExecuteDataset(sql);
    store.DataSource = dt;
    store.DataBind();
}
}
}
namespace WebForm.Controls.ClassScore
{
    public static class ClassScore
    {
        public static void BindClassScoreExtKind(this DropDownList ddl, string schoolid)
        {
            string sql = string.Format(@"select kind as [key],kind as [value]
from ClassScoreExtKind where schoolid='{0}'", schoolid);
            ddl.InitBySql(.....);
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Telerik.Web.UI;
using XSoft.CommonBiz;
using System.Web.UI.WebControls;
namespace WebForm.Controls.CommentManager
{
    public static class TelerikExtensionsForComment
    {
        public static void InitCommentKind(this RadTreeView radTreeView)
        {
            Dictionary<string, string> dicParaDetials =
ParameterManager.GetSubParameterDetials("CommentTabs");
            foreach (KeyValuePair<string, string> kvp in dicParaDetials)
            {
                RadTreeNode radNode = new RadTreeNode();
                radNode.ImageUrl = "~/images/common/view.gif";
                radNode.Text = kvp.Value;
                radNode.Value = kvp.Key;
                radTreeView.Nodes.Add();
            }
        }
        public static void InitSampleLibKind(this RadTreeView radTreeView)
        {
            Dictionary<string, string> dicParaDetials =
ParameterManager.GetSubParameterDetials("SampleLibTabs");
            foreach (KeyValuePair<string, string> kvp in dicParaDetials)
            {
                RadTreeNode radNode = new RadTreeNode();

```

```

        radNode.ImageUrl = "~/images/common/view.gif";
        radNode.Text = kvp.Value;
        radNode.Value = kvp.Key;
        radTreeView.Nodes.Add(radNode);
    }
}

public static void BindCommentKind(this DropDownList radComboBox)
{
    Dictionary<string, string> dicParaDetials =
ParameterManager.GetSubParameterDetials("CommentTabs");
    foreach (KeyValuePair<string, string> kvp in dicParaDetials)
    {
        ListItem rcbi = new ListItem();
        rcbi.Text = kvp.Value;
        rcbi.Value = kvp.Key;
        radComboBox.Items.Add();
    }
}

public static void BindSampleKind(this DropDownList radComboBox)
{
    Dictionary<string, string> dicParaDetials =
ParameterManager.GetSubParameterDetials("SampleLibTabs");
    foreach (KeyValuePair<string, string> kvp in dicParaDetials)
    {
        ListItem rcbi = new ListItem();
        rcbi.Text = kvp.Value;
        rcbi.Value = kvp.Key;
        radComboBox.Items.Add();
    }
}
}
}
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using WebForm.BusinessProcess;
using XSoft.Security;
using System.Web.UI.WebControls;
using Telerik.Web.UI;
using System.Data;
using Common.Data.Hibernate;
namespace WebForm.Controls.ComprehensivePractice
{
    public static class ComprehensivePracticeExt
    {
        public static void BindComprehensive(this RadTreeView rtv, string termId)
        {
            rtv.Nodes.Clear();
            RadTreeNode schoolNode = new

```

```

RadTreeNode(SecurityHolder.CurrentUser.SchoolInfo.SchoolName, termId);
    schoolNode.ImageUrl = "~/images/common/house.png";
    rtv.Nodes.Add(schoolNode);
    RadTreeNode csNode = new RadTreeNode("", "cs");
    csNode.ImageUrl = "~/images/common/folder-open.gif";
    schoolNode.Nodes.Add(csNode);
    BindCS(csNode, termId);
    RadTreeNode rsNode = new RadTreeNode("");
        rsNode.ImageUrl = "~/images/common/folder-open.gif";
        schoolNode.Nodes.Add(rsNode);
        BindRS(rsNode, termId);
        RadTreeNode spNode = new RadTreeNode("", "sp");
        spNode.ImageUrl = "~/images/common/folder-open.gif";
        schoolNode.Nodes.Add(spNode);
        BindSP(spNode, termId);
        schoolNode.Expanded = true;
    }

    public static void BindCS(RadTreeNode rtv, string termId)
    {
        string sql = string.Format("select id,Name,score from
EvalProKindCommunityService where schoolid='{0}' and termid='{1}'",
        SecurityHolder.CurrentUser.SchoolId,
        termId);
        DataTable table =
SessionHolder.CreateTemplate().ExecuteDataTable(sql);
        foreach (DataRow row in table.Rows)
        {
            RadTreeNode node = new RadTreeNode(row["Name"].ToString() + "(" +
row["score"].ToString() + ")", row["id"].ToString());
            node.ImageUrl = "~/images/common/folder.gif";
            rtv.Nodes.Add(node);
            BindCSItem(node, termId);
        }
        rtv.Expanded = true;
    }

    public static void BindCSItem(RadTreeNode rtv, string termId)
    {
        string sql = string.Format("select id,Name,score from
EvalProCommunityService where schoolid='{0}' and termid='{1}' and KindId='{2}'",
        SecurityHolder.CurrentUser.SchoolId,
        termId,
        rtv.Value);
        DataTable table =
SessionHolder.CreateTemplate().ExecuteDataTable(sql);
        foreach (DataRow row in table.Rows)
        {
            RadTreeNode node = new RadTreeNode(row["Name"].ToString() + "(" +
row["score"].ToString() + ")", row["id"].ToString());
            node.ImageUrl = "~/images/common/accept.png";
            rtv.Nodes.Add(node);
        }
    }

```

```

    }
}

public static void BindRS(RadTreeNode rtv, string termId)
{
    string sql = string.Format("select id,Name,score from
EvalProKindResearchStudy where schoolid='{0}' and termid='{1}'",
    SecurityHolder.CurrentUser.SchoolId,
    termId);

    DataTable table =
SessionHolder.CreateTemplate().ExecuteDataTable(sql);
    foreach (DataRow row in table.Rows)
    {
        RadTreeNode node = new RadTreeNode(row["Name"].ToString() + "(" +
row["score"].ToString() + ")", row["id"].ToString());
        node.ImageUrl = "~/images/common/folder.gif";
        rtv.Nodes.Add(node);
        BindRSItem(node, termId);
    }
    rtv.Expanded = true;
}

public static void BindRSItem(RadTreeNode rtv, string termId)
{
    string sql = string.Format("select id,Name,score from
EvalProResearchStudy where schoolid='{0}' and termid='{1}' and KindId='{2}'",
    SecurityHolder.CurrentUser.SchoolId,
    termId,
    rtv.Value);

    DataTable table =
SessionHolder.CreateTemplate().ExecuteDataTable(sql);
    foreach (DataRow row in table.Rows)
    {
        RadTreeNode node = new RadTreeNode(row["Name"].ToString() + "(" +
row["score"].ToString() + ")", row["id"].ToString());
        node.ImageUrl = "~/images/common/accept.png";
        rtv.Nodes.Add(node);
    }
}

public static void BindSP(RadTreeNode rtv, string termId)
{
    string sql = string.Format("select id,Name,score from
EvalProKindSocialPractice where schoolid='{0}' and termid='{1}'",
    SecurityHolder.CurrentUser.SchoolId,
    termId);

    DataTable table =
SessionHolder.CreateTemplate().ExecuteDataTable(sql);
    foreach (DataRow row in table.Rows)
    {
        RadTreeNode node = new RadTreeNode(row["Name"].ToString() + "(" +
row["score"].ToString() + ")", row["id"].ToString());
        node.ImageUrl = "~/images/common/folder.gif";

```

```

        rtv.Nodes.Add(node);
        BindSPItem(node, termId);
    }
    rtv.Expanded = true;
}

public static void BindSPItem(RadTreeNode rtv, string termId)
{
    string sql = string.Format("select id,Name,score from
EvalProSocialPractice where schoolid='{0}' and termid='{1}' and KindId='{2}'",
    SecurityHolder.CurrentUser.SchoolId,
    termId,
    rtv.Value);

    DataTable table =
SessionHolder.CreateTemplate().ExecuteDataTable(sql);
    foreach (DataRow row in table.Rows)
    {
        RadTreeNode node = new RadTreeNode(row["Name"].ToString() + "(" +
row["score"].ToString() + ")", row["id"].ToString());
        node.ImageUrl = "~/images/common/accept.png";
        rtv.Nodes.Add(node);
    }
}

public static void InitEvalProKindCs(this DropDownList cbx)
{
    cbx.Items.Clear();
    string sql = "select id [Key],[Name] as [Value] from
EvalProKindCommunityService where schoolid=' " +
SecurityHolder.CurrentUser.SchoolId + " ' and termid=' " +
commonFunction.getCurrentTermID() + " '";
    cbx.InitDataSet(sql);
}

public static void InitEvalProKindRs(this DropDownList cbx)
{
    cbx.Items.Clear();
    string sql = "select id [Key],[Name] as [Value] from
EvalProKindResearchStudy where schoolid=' " + SecurityHolder.CurrentUser.SchoolId
+ " ' and termid=' " + commonFunction.getCurrentTermID() + " '";
    cbx.InitDataSet(sql);
}

public static void InitEvalProKindSp(this DropDownList cbx)
{
    cbx.Items.Clear();
    string sql = "select id [Key],[Name] as [Value] from
EvalProKindSocialPractice where schoolid=' " + SecurityHolder.CurrentUser.SchoolId
+ " ' and termid=' " + commonFunction.getCurrentTermID() + " '";
    cbx.InitDataSet(.....);
}

public static void InitRole(this DropDownList cbx)
{
    cbx.Items.Clear();

```

```
        string sql = "select '1' [Key],'' as [Value] union all select '0'
[Key],'' as [Value]";
        cbx.InitDataSet(sql);
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Web.UI.WebControls;
namespace WebForm.Controls.ContractManage
{
    public static class ContractManage
    {
        public static void BindContractInfo(this DropDownList ddl, string schoolid)
        {
            ddl.Items.Clear();
            string sql = string.Format(@"select id as [key],contractname as [value]
from contractinfo where schoolid='{0}'," , schoolid);
            ddl.InitBySql(sql);
        }

        public static void BindContractInfo(this DropDownList ddl, string
schoolid, string deptid)
        {
            ddl.Items.Clear();
            string split = "";
            if (deptid != "0")
            {
                split += string.Format(@" and deptid='{0}'," ,deptid);
            }
            string sql = string.Format(@"select id as [key],contractname as [value]
from contractinfo where schoolid='{0}' {1}"," , schoolid,split);
            ddl.InitBySql(sql);
        }

        public static void BindContractCompany(this DropDownList ddl, string
schoolid)
        {
            ddl.Items.Clear();
            string sql = string.Format(@"select id as [key],companyname as [value]
from Contractcompany where schoolid='{0}'," , schoolid);
            ddl.InitBySql(" ");
        }

        public static void BindContractCompany(this DropDownList ddl, string
schoolid, bool ishasall)
        {
            ddl.BindContractCompany(schoolid);
            if (ishasall)
            {
                ddl.Items.Insert(0, new ListItem("", "0"));
            }
        }
    }
}
```



```
    }
    ddl.SelectedValue = "0";
}
public static void BindContractCurrency(this DropDownList ddl)
{
    ddl.Items.Clear();
    string sql = "select item_name as [key],item_name as [value] from
v_SysCodeManager where item_kind='114' ";
    ddl.InitBySql(sql);
}
public static void BindContractzfkind(this DropDownList ddl)
{
    ddl.Items.Clear();
    string sql = "select item_name as [key],item_name as [value] from
v_SysCodeManager where item_kind='115' ";
    ddl.InitBySql(sql);
}
public static void BindContractzftype(this DropDownList ddl)
{
    ddl.Items.Clear();
    string sql = "select item_name as [key],item_name as [value] from
v_SysCodeManager where item_kind='116' ";
    ddl.InitBySql(sql);
}
public static void BindContractState(this DropDownList ddl)
{
    ddl.Items.Clear();
    string sql = "select item_name as [key],item_name as [value] from
v_SysCodeManager where item_kind='117' ";
    ddl.InitBySql(" ");
}
public static void BindContractState(this DropDownList ddl, bool ishasall)
{
    ddl.BindContractState();
    if (ishasall)
    {
        ddl.Items.Insert(0, new ListItem("", "0"));
    }
    ddl.SelectedValue = "0";
}
public static void BindContractBuyKind(this DropDownList ddl)
{
    ddl.Items.Clear();
    string sql = "select item_name as [key],item_name as [value] from
v_SysCodeManager where item_kind='119' ";
    ddl.InitBySql(sql);
}
public static void BindContractBuyUnit(this DropDownList ddl)
{
    ddl.Items.Clear();
```

```
        string sql = "select item_name as [key],item_name as [value] from
v_SysCodeManager where item_kind='120' ";
        ddl.InitBySql(sql);
    }
}

[Serializable]
public class CategoryInfo : ICloneable, ICategoryInfo
{
    #region Member Variables
    protected string _id;
    protected string _categoryname;
    protected string _remark;
    protected string _kindid;
    protected string _property;
    protected int? _count;
    protected string _schoolid;
    protected string _categorycode;
    protected decimal? _price;
    protected string _useplace;
    protected string _place;
    protected string _manager;
    protected string _unit;
    protected string _model;
    protected string _product;
    protected string _deploy;
    protected string _indexno;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion

    #region Constructors
    public CategoryInfo() {}
    public CategoryInfo(string pId, string pCategoryName, string pRemark, string
pKindId, string pProperty, int? pCount, string pSchoolId, string pCategoryCode,
decimal? pPrice, string pUseplace, string pPlace, string pManager, string pUnit,
string pModel, string pProduct, string pDeploy, string pIndexno)
    {
        this._id = pId;
        this._categoryname = pCategoryName;
        this._remark = pRemark;
        this._kindid = pKindId;
        this._property = pProperty;
        this._count = pCount;
        this._schoolid = pSchoolId;
        this._categorycode = pCategoryCode;
        this._price = pPrice;
        this._useplace = pUseplace;
        this._place = pPlace;
        this._manager = pManager;
        this._unit = pUnit;
    }
}
```

```
this._model = pModel;
this._product = pProduct;
this._deploy = pDeploy;
this._indexno = pIndexno;
}
public CategoryInfo(string pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual string Id
{
    get { return _id; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Id", "Id value, cannot contain more than
50 characters");
        _bIsChanged |= (_id != value);
        _id = value;
    }
}
public virtual string CategoryName
{
    get { return _categoryname; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("CategoryName", "CategoryName value,
cannot contain more than 500 characters");
        _bIsChanged |= (_categoryname != value);
        _categoryname =;
    }
}
public virtual string Remark
{
    get { return _remark; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("Remark", "Remark value, cannot contain
more than 1073741823 characters");
        _bIsChanged |= ();
        _remark = value;
    }
}
public virtual string KindId
{
    get { return _kindid; }
```

```
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("KindId", "KindId value, cannot contain
more than 50 characters");
    _bIsChanged |= (_kindid != value);
    _kindid = value;
}
}
public virtual string Property
{
    get { return _property; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Property", "Property value, cannot
contain more than 50 characters");
        _bIsChanged |= (_property != value);
        _property = value;
    }
}
public virtual int? Count
{
    get { return _count; }
    set { _bIsChanged |= (_count != value); _count = value; }
}
public virtual string SchoolId
{
    get { return _schoolid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("SchoolId", "SchoolId value, cannot
contain more than 50 characters");
        _bIsChanged |= (_schoolid != value);
        _schoolid = value;
    }
}
public virtual string CategoryCode
{
    get { return _categorycode; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("CategoryCode", "CategoryCode value,
cannot contain more than 50 characters");
        _bIsChanged |= (_categorycode != value);
        _categorycode = value;
    }
}
```

```
public virtual decimal? Price
{
    get { return _price; }
    set { _bIsChanged |= (_price != value); _price = value; }
}
public virtual string Useplace
{
    get { return _useplace; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Useplace", "Useplace value, cannot
contain more than 50 characters");
        _bIsChanged |= (_useplace != value);
        _useplace = value;
    }
}
public virtual string Place
{
    get { return _place; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Place", "Place value, cannot contain
more than 50 characters");
        _bIsChanged |= (_place != value);
        _place = value;
    }
}
public virtual string Manager
{
    get { return _manager; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Manager", "Manager value, cannot
contain more than 50 characters");
        _bIsChanged |= (_manager != value);
        _manager = value;
    }
}
public virtual string Unit
{
    get { return _unit; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Unit", "Unit value, cannot contain more
than 50 characters");
        _bIsChanged |= (_unit != value);
    }
}
```

```
    _unit = value;
}
}
public virtual string Model
{
    get { return _model; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Model", "Model value, cannot contain
more than 50 characters");
        _bIsChanged |= (_model != value);
        _model = value;
    }
}
public virtual string Product
{
    get { return _product; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Product", "Product value, cannot
contain more than 50 characters");
        _bIsChanged |= (_product != value);
        _product = value;
    }
}
public virtual string Deploy
{
    get { return _deploy; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Deploy", "Deploy value, cannot contain
more than 50 characters");
        _bIsChanged |= (_deploy != value);
        _deploy = value;
    }
}
public virtual string Indexno
{
    get { return _indexno; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Indexno", "Indexno value, cannot
contain more than 50 characters");
        _bIsChanged |= (_indexno != value);
        _indexno = value;
    }
}
```

```
}
public virtual bool IsDeleted
{
    get
    {
        return _bIsDeleted;
    }
    set
    {
        _bIsDeleted = value;
    }
}
public virtual bool IsChanged
{
    get
    {
        return _bIsChanged;
    }
    set
    {
        _bIsChanged = value;
    }
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
    if( this == obj ) return true;
    CategoryInfo castObj = null;
    try
    {
        castObj = (CategoryInfo)obj;
    } catch(Exception) { return false; }
    return ( castObj != null ) &&
        ( this._id == castObj.Id );
}
public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _id.GetHashCode();
    return hash;
}
#endregion
#region ICloneable methods
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion
}
```

```

#region Custom ICollection interface for CategoryInfo
public interface ICategoryInfoCollection : ICollection
{
    CategoryInfo this[int index]{get; set; }
    void Add(CategoryInfo pCategoryInfo);
    void Clear();
}
[Serializable]
public class CategoryInfoCollection : ICategoryInfoCollection
{
    private IList<CategoryInfo> _arrayInternal;
    public CategoryInfoCollection()
    {
        _arrayInternal = new List<CategoryInfo>();
    }
    public CategoryInfoCollection( IList<CategoryInfo> pSource )
    {
        _arrayInternal = pSource;
        if(_arrayInternal == null)
        {
            _arrayInternal = new List<CategoryInfo>();
        }
    }
    public CategoryInfo this[int index]
    {
        get
        {
            return _arrayInternal[index];
        }
        set
        {
            _arrayInternal[index] = value;
        }
    }
    public int Count { get { return _arrayInternal.Count; } }
    public bool IsSynchronized { get { return false; } }
    public object SyncRoot { get { return _arrayInternal; } }
    public void CopyTo(Array array, int index){ _arrayInternal.CopyTo((CategoryInfo[])array, index); }
    public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
    public void Add(CategoryInfo pCategoryInfo) { _arrayInternal.Add(pCategoryInfo); }
    public void Clear() { _arrayInternal.Clear(); }
    public IList<CategoryInfo> GetList() { return _arrayInternal; }
}
#endregion

public class CategoryKind : ICloneable, ICategoryKind
{
    #region Member Variables
    protected string _id;

```



```
protected string _kindname;
protected string _kindcode;
protected string _parentid;
protected string _description;
protected string _schoolid;
protected bool _bIsDeleted;
protected bool _bIsChanged;
#endregion
#region Constructors
public CategoryKind() {}
public CategoryKind(string pId, string pKindName, string pKindcode, string
pParentid, string pDescription, string pSchoolId)
{
    this._id = pId;
    this._kindname = pKindName;
    this._kindcode = pKindcode;
    this._parentid = pParentid;
    this._description = pDescription;
    this._schoolid = pSchoolId;
}
public CategoryKind(string pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual string Id
{
    get { return _id; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Id", "Id value, cannot contain more than
50 characters");
        _bIsChanged |= (_id != value);
        _id = value;
    }
}
public virtual string KindName
{
    get { return _kindname; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("KindName", "KindName value, cannot
contain more than 500 characters");
        _bIsChanged |= (_kindname != value);
        _kindname = value;
    }
}
}
```

```
public virtual string Kindcode
{
    get { return _kindcode; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Kindcode", "Kindcode value, cannot
contain more than 50 characters");
        _bIsChanged |= (_kindcode != value);
        _kindcode = value;
    }
}

public virtual string Parentid
{
    get { return _parentid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Parentid", "Parentid value, cannot
contain more than 50 characters");
        _bIsChanged |= (_parentid != value);
        _parentid = value;
    }
}

public virtual string Description
{
    get { return _description; }
    set
    {
        if (value != null && value.Length > 1073741823)
            throw new ArgumentOutOfRangeException("Description", "Description value,
cannot contain more than 1073741823 characters");
        _bIsChanged |= (_description != value);
        _description = value;
    }
}

public virtual string SchoolId
{
    get { return _schoolid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("SchoolId", "SchoolId value, cannot
contain more than 50 characters");
        _bIsChanged |= (_schoolid != value);
        _schoolid = value;
    }
}

public virtual bool IsDeleted
{

```

```
get
{
return _bIsDeleted;
}
set
{
_bIsDeleted = value;
}
}
public virtual bool IsChanged
{
get
{
return _bIsChanged;
}
set
{
_bIsChanged = value;
}
}
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
if( this == obj ) return true;
CategoryKind castObj = null;
try
{
castObj = (CategoryKind)obj;
} catch(Exception) { return false; }
return ( castObj != null ) &&
( this._id == castObj.Id );
}
public override int GetHashCode()
{
int hash = 57;
hash = 27 * hash * _id.GetHashCode();
return hash;
}
}
#endregion
#region ICloneable methods
public virtual object Clone()
{
return this.MemberwiseClone();
}
}
#endregion
}
#region Custom ICollection interface for CategoryKind
public interface ICategoryKindCollection : ICollection
{
```

```

CategoryKind this[int index]{get; set; }
void Add(CategoryKind pCategoryKind);
void Clear();
}
[Serializable]
public class CategoryKindCollection : ICategoryKindCollection
{
private IList<CategoryKind> _arrayInternal;
public CategoryKindCollection()
{
_arrayInternal = new List<CategoryKind>();
}
public CategoryKindCollection( IList<CategoryKind> pSource )
{
_arrayInternal = pSource;
if(_arrayInternal == null)
{
_arrayInternal = new List<CategoryKind>();
}
}

    public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int
index){ _arrayInternal.CopyTo((CategoryKind[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(CategoryKind pCategoryKind) { _arrayInternal.Add(pCategoryKind); }
public void Clear() { _arrayInternal.Clear(); }
public IList<CategoryKind> GetList() { return _arrayInternal; }
}
#endregion
}
public class CategoryKindManager : ICloneable, ICategoryKindManager
{
#region Member Variables
protected string _id;
protected string _employeecode;
protected string _employeename;
protected string _kindid;
protected string _schoolid;
protected bool _bIsDeleted;
protected bool _bIsChanged;
#endregion
#region Constructors
public CategoryKindManager() {}
public CategoryKindManager(string pId, string pEmployeeCode, string pEmployeeName,
string pKindId, string pSchoolId)
{
this._id = pId;
this._employeecode = pEmployeeCode;

```

```
this._employeeName = pEmployeeName;
this._kindId = pKindId;
this._schoolId = pSchoolId;
}
public CategoryKindManager(string pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual string Id
{
    get { return _id; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Id", "Id value, cannot contain more than
50 characters");
        _bIsChanged |= (_id != value);
        _id = value;
    }
}
public virtual string EmployeeCode
{
    get { return _employeeCode; }
    set
    {
        if (value != null && value.Length > 100)
            throw new ArgumentOutOfRangeException("EmployeeCode", "EmployeeCode value,
cannot contain more than 100 characters");
        _bIsChanged |= (_employeeCode != value);
        _employeeCode = value;
    }
}
public virtual string EmployeeName
{
    get { return _employeeName; }
    set
    {
        if (value != null && value.Length > 100)
            throw new ArgumentOutOfRangeException("EmployeeName", "EmployeeName value,
cannot contain more than 100 characters");
        _bIsChanged |= (_employeeName != value);
        _employeeName = value;
    }
}
public virtual string KindId
{
    get { return _kindId; }
    set
```

```
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("KindId", "KindId value, cannot contain
more than 50 characters");
    _bIsChanged |= (_kindid != value);
    _kindid = value;
}
}
public virtual string SchoolId
{
    get { return _schoolid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("SchoolId", "SchoolId value, cannot
contain more than 50 characters");
        _bIsChanged |= (_schoolid != value);
        _schoolid = value;
    }
}
public virtual bool IsDeleted
{
    get
    {
        return _bIsDeleted;
    }
    set
    {
        _bIsDeleted = value;
    }
}
public virtual bool IsChanged
{
    get
    {
        return _bIsChanged;
    }
    set
    {
        _bIsChanged = value;
    }
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
    if( this == obj ) return true;
    CategoryKindManager castObj = null;
    try
    {
```

```
castObj = (CategoryKindManager)obj;
} catch(Exception) { return false; }
return ( castObj != null ) &&
( this._id == castObj.Id );
}

public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _id.GetHashCode();
    return hash;
}
#endregion

#region ICloneable methods
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion

#region Custom ICollection interface for CategoryKindManager
public interface ICategoryKindManagerCollection : ICollection
{
    CategoryKindManager this[int index]{get; set; }
    void Add(CategoryKindManager pCategoryKindManager);
    void Clear();
}

[Serializable]
public class CategoryKindManagerCollection : ICategoryKindManagerCollection
{
    private IList<CategoryKindManager> _arrayInternal;
    public CategoryKindManagerCollection()
    {
        _arrayInternal = new List<CategoryKindManager>();
    }
    public CategoryKindManagerCollection( IList<CategoryKindManager> pSource )
    {
        _arrayInternal = pSource;
        if(_arrayInternal == null)
        {
            _arrayInternal = new List<CategoryKindManager>();
        }
    }
    public CategoryKindManager this[int index]
    {
        get
        {
            return _arrayInternal[index];
        }
        set
        {

```

```
_arrayInternal[index] = value;
}
}

public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int index) { _arrayInternal.CopyTo((CategoryKindManager[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(CategoryKindManager pCategoryKindManager)
{ _arrayInternal.Add(pCategoryKindManager); }
public void Clear() { _arrayInternal.Clear(); }
public IList<CategoryKindManager> GetList() { return _arrayInternal; }
}
#endregion
}
[Serializable]
public class CategoryStock : ICloneable, ICategoryStock
{
    #region Member Variables
    protected string _id;
    protected string _stockname;
    protected string _stockcode;
    protected string _address;
    protected string _schoolid;
    protected string _remark;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion
    #region Constructors
    public CategoryStock() {}
    public CategoryStock(string pId, string pStockname, string pStockcode, string pAddress, string pSchoolid, string pRemark)
    {
        this._id = pId;
        this._stockname = pStockname;
        this._stockcode = pStockcode;
        this._address = pAddress;
        this._schoolid = pSchoolid;
        this._remark = pRemark;
    }
    public CategoryStock(string pId)
    {
        this._id = pId;
    }
    #endregion
    #region Public Properties
    public virtual string Id
    {
        get { return _id; }
    }
}
```



```
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Id", "Id value, cannot contain more than
50 characters");
    _bIsChanged |= (_id != value);
    _id = value;
}
}
public virtual string Stockname
{
    get { return _stockname; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Stockname", "Stockname value, cannot
contain more than 50 characters");
        _bIsChanged |= (_stockname != value);
        _stockname = value;
    }
}
public virtual string Stockcode
{
    get { return _stockcode; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Stockcode", "Stockcode value, cannot
contain more than 50 characters");
        _bIsChanged |= (_stockcode != value);
        _stockcode = value;
    }
}
public virtual string Address
{
    get { return _address; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Address", "Address value, cannot
contain more than 50 characters");
        _bIsChanged |= (_address != value);
        _address = value;
    }
}
public virtual string Schoolid
{
    get { return _schoolid; }
    set
    {

```

```
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Schoolid", "Schoolid value, cannot
contain more than 50 characters");
        _bIsChanged |= (_schoolid != value);
        _schoolid = value;
    }
}

public virtual string Remark
{
    get { return _remark; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Remark", "Remark value, cannot contain
more than 50 characters");
        _bIsChanged |= (_remark != value);
        _remark = value;
    }
}

public virtual bool IsDeleted
{
    get
    {
        return _bIsDeleted;
    }
    set
    {
        _bIsDeleted = value;
    }
}

public virtual bool IsChanged
{
    get
    {
        return _bIsChanged;
    }
    set
    {
        _bIsChanged = value;
    }
}

#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
    if( this == obj ) return true;
    CategoryStock castObj = null;
    try
    {
        castObj = (CategoryStock)obj;
```

```
} catch(Exception) { return false; }
return ( castObj != null ) &&
( this._id == castObj.Id );
}
public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _id.GetHashCode();
    return hash;
}
#endregion
#region ICloneable methods
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion
#region Custom ICollection interface for CategoryStock
public interface ICategoryStockCollection : ICollection
{
    CategoryStock this[int index]{get; set; }
    void Add(CategoryStock pCategoryStock);
    void Clear();
}
public class CategoryStockCollection : ICategoryStockCollection
{
    private IList<CategoryStock> _arrayInternal;
    public CategoryStockCollection()
    {
        _arrayInternal = new List<CategoryStock>();
    }
    public CategoryStockCollection( IList<CategoryStock> pSource )
    {
        _arrayInternal = pSource;
        if(_arrayInternal == null)
        {
            _arrayInternal = new List<CategoryStock>();
        }
    }
    public CategoryStock this[int index]
    {
        get
        {
            return _arrayInternal[index];
        }
        set
        {
            _arrayInternal[index] = value;
        }
    }
}
```

```

}
public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int index) { _arrayInternal.CopyTo((CategoryStock[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(CategoryStock pCategoryStock) { _arrayInternal.Add(pCategoryStock); }
public void Clear() { _arrayInternal.Clear(); }
public IList<CategoryStock> GetList() { return _arrayInternal; }
}
#endregion
}
[Serializable]
public class CategoryUse : ICloneable, ICategoryUse
{
    #region Member Variables
    protected string _id;
    protected int? _num;
    protected DateTime? _dodate;
    protected string _description;
    protected string _remark;
    protected string _state;
    protected string _categoryid;
    protected string _useplace;
    protected string _stockid;
    protected string _stockname;
    protected string _usedeptid;
    protected string _usedeptname;
    protected string _userid;
    protected string _username;
    protected string _useplaceid;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion
    #region Constructors
    public CategoryUse() {}
    public CategoryUse(string pId, int? pNum, DateTime? pDodate, string pDescription,
        string pRemark, string pState, string pCategoryid, string pUseplace, string
        pStockid, string pStockname, string pUsedeptid, string pUsedeptname, string pUserId,
        string pUsername, string pUseplaceid)
    {
        this._id = pId;
        this._num = pNum;
        this._dodate = pDodate;
        this._description = pDescription;
        this._remark = pRemark;
        this._state = pState;
        this._categoryid = pCategoryid;
    }
}

```

```
this._useplace = pUseplace;
this._stockid = pStockid;
this._stockname = pStockname;
this._usedeptid = pUsedeptid;
this._usedeptname = pUsedeptname;
this._userid = pUserid;
this._username = pUsername;
this._useplaceid = pUseplaceid;
}
public CategoryUse(string pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual string Id
{
    get { return _id; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Id", "Id value, cannot contain more than
50 characters");
        _bIsChanged |= (_id != value);
        _id = value;
    }
}
public virtual int? Num
{
    get { return _num; }
    set { _bIsChanged |= (_num != value); _num = value; }
}
public virtual DateTime? Dodate
{
    get { return _dodate; }
    set { _bIsChanged |= (_dodate != value); _dodate = value; }
}
public virtual string Description
{
    get { return _description; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("Description", "Description value,
cannot contain more than 500 characters");
        _bIsChanged |= (_description != value);
        _description = value;
    }
}
public virtual string Remark
```

```
{
get { return _remark; }
set
{
    if (value != null && value.Length > 500)
        throw new ArgumentOutOfRangeException("Remark", "Remark value, cannot contain
more than 500 characters");
    _bIsChanged |= (_remark != value);
    _remark = value;
}
}
public virtual string State
{
get { return _state; }
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("State", "State value, cannot contain
more than 50 characters");
    _bIsChanged |= (_state != value);
    _state = value;
}
}
public virtual string Categoryid
{
get { return _categoryid; }
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Categoryid", "Categoryid value, cannot
contain more than 50 characters");
    _bIsChanged |= (_categoryid != value);
    _categoryid = value;
}
}
public virtual string Useplace
{
get { return _useplace; }
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Useplace", "Useplace value, cannot
contain more than 50 characters");
    _bIsChanged |= (_useplace != value);
    _useplace = value;
}
}
public virtual string Stockid
{
get { return _stockid; }
```

```
set
{
    if (value != null && value.Length > 50)
        throw new ArgumentOutOfRangeException("Stockid", "Stockid value, cannot
contain more than 50 characters");
    _bIsChanged |= (_stockid != value);
    _stockid = value;
}
}
public virtual string Stockname
{
    get { return _stockname; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Stockname", "Stockname value, cannot
contain more than 50 characters");
        _bIsChanged |= (_stockname != value);
        _stockname = value;
    }
}
public virtual string Usedeptid
{
    get { return _usedeptid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Usedeptid", "Usedeptid value, cannot
contain more than 50 characters");
        _bIsChanged |= (_usedeptid != value);
        _usedeptid = value;
    }
}
public virtual string Usedeptname
{
    get { return _usedeptname; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Usedeptname", "Usedeptname value,
cannot contain more than 50 characters");
        _bIsChanged |= (_usedeptname != value);
        _usedeptname = value;
    }
}
public virtual string Userid
{
    get { return _userid; }
    set
    {

```

```
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Userid", "Userid value, cannot contain
more than 50 characters");
        _bIsChanged |= (_userid != value);
        _userid = value;
    }
}

public virtual string Username
{
    get { return _username; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Username", "Username value, cannot
contain more than 50 characters");
        _bIsChanged |= (_username != value);
        _username = value;
    }
}

public virtual string Useplaceid
{
    get { return _useplaceid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Useplaceid", "Useplaceid value, cannot
contain more than 50 characters");
        _bIsChanged |= (_useplaceid != value);
        _useplaceid = value;
    }
}

public virtual bool IsDeleted
{
    get
    {
        return _bIsDeleted;
    }
    set
    {
        _bIsDeleted = value;
    }
}

public virtual bool IsChanged
{
    get
    {
        return _bIsChanged;
    }
    set
    {
```



```
_bIsChanged = value;
}
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
    if( this == obj ) return true;
    CategoryUse castObj = null;
    try
    {
        castObj = (CategoryUse)obj;
    } catch(Exception) { return false; }
    return ( castObj != null ) &&
        ( this._id == castObj.Id );
}
public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _id.GetHashCode();
    return hash;
}
#endregion
#region ICloneable methods
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion
}
#region Custom ICollection interface for CategoryUse
public interface ICategoryUseCollection : ICollection
{
    CategoryUse this[int index]{get; set; }
    void Add(CategoryUse pCategoryUse);
    void Clear();
}
[Serializable]
public class CategoryUseCollection : ICategoryUseCollection
{
    private IList<CategoryUse> _arrayInternal;
    public CategoryUseCollection()
    {
        _arrayInternal = new List<CategoryUse>();
    }
    public CategoryUseCollection( IList<CategoryUse> pSource )
    {
        _arrayInternal = pSource;
        if(_arrayInternal == null)
        {
```

```

_arrayInternal = new List<CategoryUse>();
}
}
public CategoryUse this[int index]
{
    get
    {
        return _arrayInternal[index];
    }
    set
    {
        _arrayInternal[index] = value;
    }
}

public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int index) { _arrayInternal.CopyTo((CategoryUse[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(CategoryUse pCategoryUse) { _arrayInternal.Add(pCategoryUse); }
public void Clear() { _arrayInternal.Clear(); }
public IList<CategoryUse> GetList() { return _arrayInternal; }
}
#endregion
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using Telerik.Web.UI;
using XSoft.Security;
using XSoft.Utility;
using WebForm.BusinessProcess;
using System.Web.UI.WebControls;
using System.Data;
using Common.Data.Hibernate;
namespace WebForm.Controls.CourseScheduling
{
    public static class RadComboBoxExtensions
    {
        public static void InitClassLessonPlace(this DropDownList cbx, string termid)
        {
            string sql = "select id [Key],placename as [Value] from ClassLessonPlace where schoolid='" + SecurityHolder.CurrentUser.SchoolId + "' and termid='" + termid + "'";
            cbx.InitBySql(sql);
        }

        public static void InitModuleBySubject(this Ext.Net.Store store, string

```

```

subjectid)
{
    string sql = string.Format(@"select id as [key],[abbreviation] [value]
from creditmodule where schoolid='{0}' and subjectid='{1}' and [name]<>'',"
SecurityHolder.CurrentUser.SchoolId, subjectid);
    store.InitDataSet(sql);
}

public static void InitAllModule(this Ext.Net.Store store)
{
    string sql = string.Format(@"select id as [key],[abbreviation] [value]
from creditmodule where schoolid='{0}' and type='0' and [name]<>'',"
SecurityHolder.CurrentUser.SchoolId);
    store.InitDataSet(sql);
}

public static void InitModuleBySubject(this Ext.Net.ComboBox cbx, string
subjectid)
{
    string sql = string.Format(@"select id as [key],[abbreviation] [value]
from creditmodule where schoolid='{0}' and subjectid='{1}' and type='0' and
[name]<>''," SecurityHolder.CurrentUser.SchoolId, subjectid);
    cbx.InitDataSet(sql);
}

public static void InitModuleMultiCombo(this Ext.Net.MultiCombo cbx)
{
    cbx.Items.Clear();
    string sql = string.Format(@"select id as [key],[abbreviation] [value]
from creditmodule where schoolid='{0}' and type='0' and [name]<>'I' order by
id,abbreviation", SecurityHolder.CurrentUser.SchoolId);
    DataTable dt = SessionHolder.CreateTemplate().ExecuteDataTable(sql);
    foreach (DataRow row in dt.Rows)
    {
        cbx.Items.Add(new Ext.Net.ListItem(row["value"].ToString(),
row["id"].ToString()));
    }
}

public static void InitCurrentClassPlace(this DropDownList ddl)
{
    string sql = "select id [Key],placename as [Value] from
ClassLessonPlace where schoolid='" + SecurityHolder.CurrentUser.SchoolId + "' and
termid='" + commonFunction.getCurrentTermID() + "' and iscurrent='1'";
    ddl.InitBySql(sql);
}

public static void BindClassPlace(this DropDownList ddl)
{
    string sql = "select id [Key],placename as [Value] from
ClassLessonPlace where schoolid='" + SecurityHolder.CurrentUser.SchoolId + "' and
termid='" + commonFunction.getCurrentTermID() + "'";
    ddl.InitBySql(sql);
}

public static void InitClassLessonTemp(this RadComboBox cbx, string

```

```

placeId)
{
    if (placeId.IsNull()) return;
    string sql = "select a.classid [Key], a.classname [Value] from classinfo
a,classlesson temp b where a.classid=b.classid and b.placeid=" + placeId + " group
by a.classid ,a.classname";
    cbx.InitBySql(sql);
}

public static void BindTakeLessonKind(this DropDownList ddl)
{
    string sql = "select id [Key],Name as [Value] from TakeLessonKind where
schoolid=' " + SecurityHolder.CurrentUser.SchoolId + "' ";
    ddl.InitBySql(sql);
}
}

public static void InitFieldSubjectModule(this RadTreeView rvtModule, string
termId)
{
    rvtModule.Nodes.Clear();
    List<OrmLib.CreditField> cfs =
SessionHolder.Current.Linq<OrmLib.CreditField>().Where(c => c.SchoolId ==
SecurityHolder.CurrentUser.SchoolId).ToList();
    RadTreeNode rootnode = new RadTreeNode(
RadTreeNode(SecurityHolder.CurrentUser.SchoolInfo.SchoolName, termId);
    rootnode.ImageUrl = "~/images/common/house.png";
    rvtModule.Nodes.Add(rootnode);
    foreach (OrmLib.CreditField cf in cfs)
    {
        RadTreeNode tn = new RadTreeNode();
        tn.Text = cf.Name + "";
        tn.Value = cf.Id.ToString();
        tn.ImageUrl = "~/images/common/folder-open.gif";
        InitSubject(tn, cf.Id.ToString(), termId);
        rootnode.Nodes.Add(tn);
    }
    if (rvtModule.Nodes.Count <= 0) return;
    rvtModule.Nodes[0].Expanded = true;
}

namespace WebForm.Controls.DeptManger
{
    public static class DeptExtend
    {
        public static void BindDept(this RadTreeView tv)
        {
            tv.BindDept("-1");
        }
        public static void BindDept(this RadTreeView tv, string pdeptid)
        {
            tv.Nodes.Clear();

```

```

        string sql = string.Format(@"select depid,deptname from department
where depid='{0}', pdeptid);
        DataTable dt = SessionHolder.CreateTemplate().ExecuteDataTable(sql);
        if (dt == null || dt.Rows.Count == 0)
        {
            return;
        }
        RadTreeNode tn = new RadTreeNode();
        tn.Text = dt.Rows[0]["deptname"].ToString();
        tn.Value = dt.Rows[0]["depid"].ToString();
        tv.Nodes.Add(tn);
        sql = string.Format(@"with myT as(select depid, deptname, parentid from
department ),
                                myT2 as(select * from myT where
parentid='{0}'
                                union all
                                select myT.* from myT2 inner join myT
                                on myT2.depid=myT.parentid
                                )
                                select depid,deptname,parentid from
myT2", pdeptid);
        dt = SessionHolder.CreateTemplate().ExecuteDataTable(sql);
        BindSunDepartment(tn, dt);
        tv.ExpandAllNodes();
    }
    private static void BindSunDepartment(RadTreeNode tn, DataTable dt)
    {
        foreach (DataRow dr in dt.Select(string.Format("parentid='{0}'",
tn.Value)))
        {
            RadTreeNode stn = new RadTreeNode();
            stn.Text = dr["deptname"].ToString();
            stn.Value = dr["depid"].ToString();
            tn.Nodes.Add(stn);
            BindSunDepartment(stn, dt);
        }
    }
}
[Serializable]
public class ClassAllCocktail : ICloneable, IClassAllCocktail
{
    #region Member Variables
    protected int _id;
    protected string _usercode;
    protected string _username;
    protected string _usertype;
    protected string _maxim;
    protected DateTime? _date;
    protected string _classid;
    protected string _schoolid;
    protected bool _bIsDeleted;

```

```
protected bool _bIsChanged;
#endregion
#region Constructors
public ClassAllCocktail() {}
public ClassAllCocktail(string pUserCode, string pUserName, string pUserType,
string pMaxim, DateTime? pDate, string pClassId, string pSchoolId)
{
    this._usercode = pUserCode;
    this._username = pUserName;
    this._usertype = pUserType;
    this._maxim = pMaxim;
    this._date = pDate;
    this._classid = pClassId;
    this._schoolid = pSchoolId;
}
public ClassAllCocktail(int pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual int Id
{
    get { return _id; }
    set { _bIsChanged |= (_id != value); _id = value; }
}
public virtual string UserCode
{
    get { return _usercode; }
    set
    {
        if (value != null && value.Length > 100)
            throw new ArgumentOutOfRangeException("UserCode", "UserCode value, cannot
contain more than 100 characters");
        _bIsChanged |= ();
        _usercode = value;
    }
}
public virtual string UserName
{
    get { return _username; }
    set
    {
        if (value != null && value.Length > 100)
            throw new ArgumentOutOfRangeException("UserName", "UserName value, cannot
contain more than 100 characters");
        _bIsChanged |= (_username != value);
        _username = value;
    }
}
}
```

```
public virtual string UserType
{
    get { return _usertype; }
    set
    {
        if (value != null && value.Length > 100)
            throw new ArgumentOutOfRangeException("UserType", "UserType value, cannot
contain more than 100 characters");
        _bIsChanged |= (_usertype != value);
        _usertype = value;
    }
}

public virtual string Maxim
{
    get { return _maxim; }
    set
    {
        if (value != null && value.Length > 2147483647)
            throw new ArgumentOutOfRangeException("Maxim", "Maxim value, cannot contain
more than 2147483647 characters");
        _bIsChanged |= (_maxim != value);
        _maxim = value;
    }
}

public virtual DateTime? Date
{
    get { return _date; }
    set { _bIsChanged |= (_date != value); _date = value; }
}

public virtual string ClassId
{
    get { return _classid; }
    set
    {
        if (value != null && value.Length > 100)
            throw new ArgumentOutOfRangeException("ClassId", "ClassId value, cannot
contain more than 100 characters");
        _bIsChanged |= ();
        _classid = value;
    }
}

public class ClassBigEvent : ICloneable, IClassBigEvent
{
    #region Member Variables
    protected int _id;
    protected string _classid;
    protected string _termid;
    protected string _contenet;
    protected string _schoolid;
    protected string _title;
```

```
protected DateTime? _date;
protected string _usercode;
protected string _username;
protected bool _bIsDeleted;
protected bool _bIsChanged;
#endregion
#region Constructors
public ClassBigEvent() {}
public ClassBigEvent(string pClassId, string pTermId, string pConenent, string
pSchoolId, string pTitle, DateTime? pDate, string pUserCode, string pUserName)
{
    this._classid = pClassId;
    this._termid = pTermId;
    this._conenent = pConenent;
    this._schoolid = pSchoolId;
    this._title = pTitle;
    this._date = pDate;
    this._usercode = pUserCode;
    this._username = pUserName;
}
public ClassBigEvent(int pId)
{
    this._id = pId;
}
#endregion
#region Public Properties
public virtual int Id
{
    get { return _id; }
    set { _bIsChanged |= (_id != value); _id = value; }
}
public virtual string ClassId
{
    get { return _classid; }
    set
    {
        if (value != null && value.Length > 32)
            throw new ArgumentOutOfRangeException("ClassId", "ClassId value, cannot
contain more than 32 characters");
        _bIsChanged |= (_classid != value);
        _classid = value;
    }
}
public virtual string TermId
{
    get { return _termid; }
    set
    {
        if (value != null && value.Length > 32)
            throw new ArgumentOutOfRangeException("TermId", "TermId value, cannot contain
```



```
more than 32 characters");
    _bIsChanged |= (_termid != value);
    _termid = value;
}
}
public virtual string Contenet
{
    get { return _contentet; }
    set
    {
        if (value != null && value.Length > 2000)
            throw new ArgumentOutOfRangeException("Contentet", "Contentet value, cannot
contain more than 2000 characters");
        _bIsChanged |= (_contentet != value);
        _contentet = value;
    }
}
public virtual string SchoolId
{
    get { return _schoolid; }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("SchoolId", "SchoolId value, cannot
contain more than 50 characters");
        _bIsChanged |= (_schoolid != value);
        _schoolid = value;
    }
}
public virtual string Title
{
    get { return _title; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("Title", "Title value, cannot contain
more than 500 characters");
        _bIsChanged |= (_title != value);
        _title = value;
    }
}
public virtual DateTime? Date
{
    get { return _date; }
    set { _bIsChanged |= (_date != value); _date = value; }
}
public virtual string UserCode
{
    get { return _usercode; }
    set
```

```
{
    if (value != null && value.Length > 500)
        throw new ArgumentOutOfRangeException("UserCode", "UserCode value, cannot
contain more than 500 characters");
    _bIsChanged |= (_usercode != value);
    _usercode = value;
}
}
public virtual string UserName
{
    get { return _username; }
    set
    {
        if (value != null && value.Length > 500)
            throw new ArgumentOutOfRangeException("UserName", "UserName value, cannot
contain more than 500 characters");
        _bIsChanged |= (_username != value);
        _username = value;
    }
}
}
public class ClassJournal : ICloneable, IClassJournal
{
    #region Member Variables
    protected int _cjid;
    protected int? _ctid;
    protected int? _studentcount;
    protected int? _girlcount;
    protected int? _minoritycount;
    protected string _monitorname;
    protected string _classhonorary;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion
    #region Constructors
    public ClassJournal() {}
    public ClassJournal(int? pCtId, int? pStudentCount, int? pGirlCount, int?
pMinorityCount, string pMonitorName, string pClassHonorary)
    {
        this._ctid = pCtId;
        this._studentcount = pStudentCount;
        this._girlcount = pGirlCount;
        this._minoritycount = pMinorityCount;
        this._monitorname = pMonitorName;
        this._classhonorary = pClassHonorary;
    }
    #endregion
    #region Public Properties
    public virtual int CjId
    {
        get { return _cjid; }
```

```
set { _bIsChanged |= (_cjid != value); _cjid = value; }
}
public virtual int? CtId
{
get { return _ctid; }
set { _bIsChanged |= (_ctid != value); _ctid = value; }
}
public virtual int? StudentCount
{
get { return _studentcount; }
set { _bIsChanged |= (_studentcount != value); _studentcount = value; }
}
public virtual int? GirlCount
{
get { return _girlcount; }
set { _bIsChanged |= (_girlcount != value); _girlcount = value; }
}
public virtual int? MinorityCount
{
get { return _minoritycount; }
set { _bIsChanged |= (_minoritycount != value); _minoritycount = value; }
}
public virtual string MonitorName
{
get { return _monitorname; }
set
{
if (value != null && value.Length > 10)
throw new ArgumentOutOfRangeException("MonitorName", "MonitorName value,
cannot contain more than 10 characters");
_bIsChanged |= (_monitorname != value);
_monitorname = value;
}
}
public virtual string ClassHonorary
{
get { return _classhonorary; }
set
{
if (value != null && value.Length > 100)
throw new ArgumentOutOfRangeException("ClassHonorary", "ClassHonorary value,
cannot contain more than 100 characters");
_bIsChanged |= (_classhonorary != value);
_classhonorary = value;
}
}
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{

```

```
if( this == obj ) return true;
ClassJournal castObj = null;
try
{
    castObj = (ClassJournal)obj;
} catch(Exception) { return false; }
return ( castObj != null ) &&
( this._cjid == castObj.CjId );
}

public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _cjid.GetHashCode();
    return hash;
}
#endregion

#region ICloneable methods
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion

#region Custom ICollection interface for ClassJournal
public interface ICollection : ICollection
{
    ClassJournal this[int index]{get; set; }
    void Add(ClassJournal pClassJournal);
    void Clear();
}
[Serializable]
public class ClassJournalCollection : ICollection
{
    private IList<ClassJournal> _arrayInternal;
    public ClassJournalCollection()
    {
        _arrayInternal = new List<ClassJournal>();
    }
    public ClassJournalCollection( IList<ClassJournal> pSource )
    {
        _arrayInternal = pSource;
        if(_arrayInternal == null)
        {
            _arrayInternal = new List<ClassJournal>();
        }
    }
    public ClassJournal this[int index]
    {
        get
        {
```

```

return _arrayInternal[index];
}

set
{
_arrayInternal[index] = value;
}
}

public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int index) { _arrayInternal.CopyTo((ClassJournal[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(ClassJournal pClassJournal) { _arrayInternal.Add(pClassJournal); }
public void Clear() { _arrayInternal.Clear(); }
public IList<ClassJournal> GetList() { return _arrayInternal; }
}

#endregion
}

namespace WebForm.Controls.DeptManger
{
    public static class DeptExtend
    {
        public static void BindDept(this RadTreeView tv)
        {
            tv.BindDept("-1");
        }
        public static void BindDept(this RadTreeView tv, string pdeptid)
        {
            tv.Nodes.Clear();
            string sql = string.Format(@"select depid,deptname from department
where depid='{0}',", pdeptid);
            DataTable dt = SessionHolder.CreateTemplate().ExecuteDataTable(sql);
            if (dt == null || dt.Rows.Count == 0)
            {
                return;
            }
            RadTreeNode tn = new RadTreeNode();
            tn.Text = dt.Rows[0]["deptname"].ToString();
            tn.Value = dt.Rows[0]["depid"].ToString();
            tv.Nodes.Add(tn);
            sql = string.Format(@"with myT as(select depid,deptname,parentid from
department ),
myT2 as(select * from myT where
parentid='{0}',
union all
select myT.* from myT2 inner join myT
on myT2.depid=myT.parentid
)
select depid,deptname,parentid from

```

```

myT2", pdeptid);
        dt = SessionHolder.CreateTemplate().ExecuteDataTable(sql);
        BindSunDepartment(tn, dt);
        tv.ExpandAllNodes();
    }
    private static void BindSunDepartment(RadTreeNode tn, DataTable dt)
    {
        foreach (DataRow dr in dt.Select(string.Format("parentid='{0}'",
tn.Value)))
        {
            RadTreeNode stn = new RadTreeNode();
            stn.Text = dr["deptname"].ToString();
            stn.Value = dr["depid"].ToString();
            tn.Nodes.Add(stn);
            BindSunDepartment(stn, dt);
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Web.UI.WebControls;
using XSoft.Security;
namespace WebForm.Controls.DormitoryMgr
{
    public static class DormitoryExt
    {
        public static void BindDormitoryKind(this DropDownList ddl)
        {
            ddl.Items.Clear();
            string sql = "select Id as [Key], [Name] as [Value] from DormitoryMgrKind
where SchoolId='" + SecurityHolder.CurrentUser.SchoolId + "'";
            if (SecurityHolder.CurrentUser.UserType == UserType.Teacher)
            {
                sql += " and charindex('" + SecurityHolder.CurrentUser.Id +
"", usercode)>0";
            }
            ddl.InitBySql(sql);
        }
        public static void BindDormitoryAllKind(this DropDownList ddl)
        {
            ddl.Items.Clear();
            string sql = "select Id as [Key], [Name] as [Value] from DormitoryMgrKind
where SchoolId='" + SecurityHolder.CurrentUser.SchoolId + "'";
            ddl.InitBySql(sql);
        }
    }
}
using System;
using System.Collections.Generic;

```

```

using System.Linq;
using System.Text;
using Telerik.Web.UI;
using XSoft.Security;
using System.Web.UI.WebControls;
using Common.Data.Hibernate;
using XSoft.CommonBiz;
using WebForm.Common;
using NHibernate.Linq;
namespace WebForm.Controls.EcManager
{
    public static class RadComboBoxExtensions
    {
        public static void BindEduTermInfo(this RadComboBox radCombox, string
educationId, bool isSelCurrentTerm)
        {
            string sql = string.Format(@"select termid as [key], termName as [value]
from TermInfo_edu where educationid='{0}'
                                order by schoolyear,term", educationId);
            radCombox.InitBySql(sql);
            if (isSelCurrentTerm)
            {
                radCombox.SelectedValue =
XSoft.CommonBiz.EduBaseManager.GetCurEduTermId(educationId);
            }
        }

        public static void BindEduTermInfo(this DropDownList ddl, string
educationId, bool isSelCurrentTerm)
        {
            string sql = string.Format(@"select termid as [key], termName as [value]
from TermInfo_edu where educationid='{0}'
                                order by schoolyear,term", educationId);
            ddl.InitBySql(sql);
            if (isSelCurrentTerm)
            {
                ddl.SelectedValue =
XSoft.CommonBiz.EduBaseManager.GetCurEduTermId(educationId);
            }
        }

        public static void BindBaseGradeInfo(this RadComboBox radCombox)
        {
            string sql = string.Format(@"select a.sectionname+a.gradename as
value,
                                cast(a.section
                                as
varchar)+cast(a.gradeindex as varchar) as text from GradeReportTemp a
                                group
                                by
a.sectionname, a.gradename, a.section, a.gradeindex
                                order by a.section,a.gradeindex");
            DataMorph.CreateInstance.InitRadComboBox(ref radCombox, sql);
        }
    }
}

```

```

        public static void BindBaseGradeInfo(this DropDownList radCombox)
        {
            string sql = string.Format(@"select a.sectionname+a.gradename as text,
                                         cast(a.section          as
varchar)+cast(a.gradeindex as varchar) as value from GradeReportTemp a
                                         group
                                         by
a.sectionname, a.gradename, a.section, a.gradeindex
                                         order by a.section, a.gradeindex");
            DataMorph.CreateInstance.InitRadComboBox(ref radCombox, sql);
            public static void BindEduSchoolList(this CheckBoxLayout cbl, string
educationid, string section)
            {
                string          schoolstr          =
XSoft.CommonBiz.EduBaseManager.GetSchoolStrByEdu(educationid);
                string  sql  =  string.Format(@"select  b.schoolid  as
[key], b.schoolname as [value]
                from  schoolinfo b where b.schoolid in (
            {
                0
            }
            ) and b.schoolid in
                (select  schoolid  from  schoolsection  where
section='{1}' and isused='1' )", schoolstr, section);
                cbl.InitBySql(sql);
            }
            public static void BindEduSchool(this DropDownList cbl, string
educationid)
            {
                string          schoolstr          =
XSoft.CommonBiz.EduBaseManager.GetSchoolStrByEdu(educationid);
                string sql = string.Format(@"select b.schoolid as [key], b.schoolname
as [value]
                from  schoolinfo b where b.schoolid in (
            {
                0
            }
            ) ", schoolstr);
                cbl.InitBySql(sql);
            }
            public static void BindEduSchoolList(this RadComboBox cbl, string
educationid)
            {
                string          allschoolstr          =
XSoft.CommonBiz.EduBaseManager.GetSchoolStrByEdu(educationid);
                string sql = string.Format(@"select b.schoolid as [key], b.schoolname
as [value]
                from  schoolinfo b where b.schoolid  in(
            {
                0
            }

```



```

    ), allschoolstr);
        cbl.InitBySql(sql);
    }
    public static void BindEduExam(this RadComboBox cbl, string termid)
    {
        string sql = string.Format(@"select examid as [key], examname as [value]
from examname_edu where termid='{0}' ", termid);
        cbl.InitBySql(sql);
    }
    public static void BindEduExam(this DropDownList cbl, string termid)
    {
        string sql = string.Format(@"select examid as [key], examname as [value]
from examname_edu where termid='{0}' ", termid);
        cbl.InitBySql(sql);
    }
    public static void BindEduSubject(this DropDownList cbl, string eduid)
    {
        string sql = string.Format(@"select code as [key], name as [value] from
Dm_Subject_Education where EducationId='{0}' order by code", eduid);
        cbl.InitBySql(sql);
    }
    public static void BindEduSubject(this CheckBoxList cbl, string eduid)
    {
        string sql = string.Format(@"select code as [key], name as [value] from
Dm_Subject_Education where EducationId='{0}' order by code", eduid);
        cbl.InitBySql(sql);
    }
    public static void BindEduExamSchool(this RadComboBox cbl, string examid)
    {
        string sql = string.Format(@"select a.schoolid as [key], b.schoolname
as[value] from  dbo.ExamSchoolDetails_edu a, schoolinfo b where a.examid='{0}' and
a.schoolid=b.schoolid", examid);
        cbl.InitBySql(sql);
        cbl.Items.Insert(0, new RadComboBoxItem("", "-1"));
    }
    public static void BindEduExamSchool(this DropDownList cbl, string examid)
    {
        string sql = string.Format(@"select a.schoolid as [key], b.schoolname
as[value] from  dbo.ExamSchoolDetails_edu a, schoolinfo b where a.examid='{0}' and
a.schoolid=b.schoolid", examid);
        cbl.InitBySql(sql);
        cbl.Items.Insert(0, new ListItem("", "-1"));
    }
    public static void BindEduExamSubject(this RadComboBox cbl, string examid,
bool isall)
    {
        string sql = string.Format(@"select a.subjectcode [key], b.[name]
[value] from ExamSubjectDetails_edu a,
                                dm_subject_education b, examname_edu
c where a.examid=c.examid and a.examid='{0}'

```

```

                                and b.educationid=c.educationid and
a.subjectcode=b.code", examid);
    cbl.InitBySql(sql);
    if (isall)
    {
        cbl.Items.Insert(0, new RadComboBoxItem(", "*));
    }
}

public static void BindEduExamSubject(this DropDownList cbl, string examid, bool
isall)
{
    string sql = string.Format(@"select a.subjectcode [key],b.[name] [value] from
ExamSubjectDetails_edu a,
                                dm_subject_education b ,examname_edu c
where a.examid=c.examid and a.examid='{0}'
                                and    b.educationid=c.educationid    and
a.subjectcode=b.code", examid);
    cbl.InitBySql(sql);
    if (isall)
    {
        cbl.Items.Insert(0, new ListItem("", "*"));
    }
}
}
}
}

[Serializable]
public class ClassLesson : ICloneable, IClassLesson
{
    #region Member Variables
    protected int _clid;
    protected int? _dayofweek;
    protected int? _numberoflesson;
    protected string _subjectid;
    protected string _subjectname;
    protected string _moduleid;
    protected string _modulename;
    protected string _teacherid;
    protected string _teachername;
    protected DateTime? _startdate;
    protected string _venue;
    protected byte? _ptimes;
    protected string _fix;
    protected byte? _intime;
    protected string _studentassessment;
    protected string _termid;
    protected string _classid;
    protected string _code;
    protected bool _bIsDeleted;
    protected bool _bIsChanged;
    #endregion
}

```

```
#region Constructors
public ClassLesson()
{
}

public ClassLesson(int? pDayOfWeek, int? pNumberOfLesson, string pSubjectId,
string pSubjectname, string pModuleid, string pModulename, string pTeacherId,
string pTeacherName, DateTime? pStartdate, string pVenue, byte? pPtimes, string
pFix, byte? pIntime, string pStudentAssessment, string pTermid, string pClassid,
string pCode)
{
this._dayofweek = pDayOfWeek;
this._numberoflesson = pNumberOfLesson;
this._subjectid = pSubjectId;
this._subjectname = pSubjectname;
this._moduleid = pModuleid;
this._modulename = pModulename;
this._teacherid = pTeacherId;
this._teachername = pTeacherName;
this._startdate = pStartdate;
this._venue = pVenue;
this._ptimes = pPtimes;
this._fix = pFix;
this._intime = pIntime;
this._studentassessment = pStudentAssessment;
this._termid = pTermid;
this._classid = pClassid;
this._code = pCode;
}

public ClassLesson(int pClId)
{
this._clid = pClId;
}

#endregion
#region Public Properties
public virtual int ClId
{
get { return _clid;
}
set
{
_bIsChanged |= (_clid != value);
_clid = value;
}
}

public virtual int? DayOfWeek
{
get { return _dayofweek;
}
set
{

```

```
_bIsChanged |= (_dayofweek != value);
_dayofweek = value;
}
}

public virtual int? NumberOfLesson
{
    get { return _numberoflesson; }
    set
    {
        _bIsChanged |= (_numberoflesson != value);
        _numberoflesson = value;
    }
}

public virtual string SubjectId
{
    get
    {
        return _subjectid;
    }
    set
    {
        if (value != null && value.Length > 1000)
            throw new ArgumentOutOfRangeException("SubjectId", "SubjectId value, cannot
            contain more than 1000 characters");
        _bIsChanged |= (_subjectid != value);
        _subjectid = value;
    }
}

public virtual string Subjectname
{
    get
    {
        return _subjectname;
    }
    set
    {
        if (value != null && value.Length > 1000)
            throw new ArgumentOutOfRangeException("Subjectname", "Subjectname value,
            cannot contain more than 1000 characters");
        _bIsChanged |= (_subjectname != value);
        _subjectname = value;
    }
}

public virtual string Moduleid
{
    get
    {
        return _moduleid;
    }
}
```

```
set
{
if (value != null && value.Length > 50)
    throw new ArgumentOutOfRangeException("Moduleid", "Moduleid value, cannot
contain more than 50 characters");
_bIsChanged |= (_moduleid != value);
_moduleid = value;
}
}

public virtual string Modulename
{
get
{
return _modulename;
}
set
{
if (value != null && value.Length > 50)
    throw new ArgumentOutOfRangeException("Modulename", "Modulename value, cannot
contain more than 50 characters");
_bIsChanged |= (_modulename != value);
_modulename = value;
}
}

public virtual string TeacherId
{
get
{
return _teacherid;
}
set
{
if (value != null && value.Length > 1000)
    throw new ArgumentOutOfRangeException("TeacherId", "TeacherId value, cannot
contain more than 1000 characters");
_bIsChanged |= (_teacherid != value);
_teacherid = value;
}
}

public virtual string TeacherName
{
get
{
return _teachername;
}
set
{
if (value != null && value.Length > 1000)
    throw new ArgumentOutOfRangeException("TeacherName", "TeacherName value,
cannot contain more than 1000 characters");
```

```
_bIsChanged |= (_teachername != value);
_teachername = value;
}
}

public virtual DateTime? Startdate
{
    get
    {
        return _startdate;
    }
    set
    {
        _bIsChanged |= (_startdate != value);
        _startdate = value;
    }
}

public virtual string Venue
{
    get
    {
        return _venue;
    }
    set
    {
        if (value != null && value.Length > 255)
            throw new ArgumentOutOfRangeException("Venue", "Venue value, cannot contain more than 255 characters");
        _bIsChanged |= (_venue != value);
        _venue = value;
    }
}

public virtual byte? Ptimes
{
    get { return _ptimes; }
    set
    {
        _bIsChanged |= (_ptimes != value);
        _ptimes = value;
    }
}

public virtual string Fix
{
    get
    {
        return _fix;
    }
    set
    {
        if (value != null && value.Length > 50)
```

```
        throw new ArgumentOutOfRangeException("Fix", "Fix value, cannot contain more
than 50 characters");
    _bIsChanged |= (_fix != value);
    _fix = value;
}
}

public virtual byte? Intime
{
    get { return _intime; }
    set
    {
        _bIsChanged |= (_intime != value);
        _intime = value;
    }
}

public virtual string StudentAssessment
{
    get
    {
        return _studentassessment;
    }
    set
    {
        if (value != null && value.Length > 255)
            throw new ArgumentOutOfRangeException("StudentAssessment",
"StudentAssessment value, cannot contain more than 255 characters");
        _bIsChanged |= (_studentassessment != value);
        _studentassessment = value;
    }
}

public virtual string Termid
{
    get
    {
        return _termid;
    }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Termid", "Termid value, cannot contain
more than 50 characters");
        _bIsChanged |= (_termid != value);
        _termid = value;
    }
}

public virtual string Classid
{
    get
    {
```

```
return _classid;
}
set
{
if (value != null && value.Length > 50)
    throw new ArgumentOutOfRangeException("Classid", "Classid value, cannot
contain more than 50 characters");
_bIsChanged |= (_classid != value);
_classid = value;
}
}
public virtual string Code
{
get
{
return _code;
}
set
{
if (value != null && value.Length > 50)
    throw new ArgumentOutOfRangeException("Code", "Code value, cannot contain more
than 50 characters");
_bIsChanged |= (_code != value);
_code = value;
}
}
public virtual bool IsDeleted
{
get
{
return _bIsDeleted;
}
set
{
_bIsDeleted = value;
}
}
public virtual string Product
{
get
{
return _product;
}
set
{
if (value != null && value.Length > 50)
    throw new ArgumentOutOfRangeException("Product", "Product value, cannot
contain more than 50 characters");
_bIsChanged |= (_product != value);
_product = value;
}
```



```
}
}
public virtual string Deploy
{
    get
    {
        return _deploy;
    }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Deploy", "Deploy value, cannot contain
more than 50 characters");
        _bIsChanged |= (_deploy != value);
        _deploy = value;
    }
}
public virtual string Indexno
{
    get
    {
        return _indexno;
    }
    set
    {
        if (value != null && value.Length > 50)
            throw new ArgumentOutOfRangeException("Indexno", "Indexno value, cannot
contain more than 50 characters");
        _bIsChanged |= (_indexno != value);
        _indexno = value;
    }
}
public virtual bool IsDeleted
{
    get
    {
        return _bIsDeleted;
    }
    set
    {
        _bIsDeleted = value;
    }
}
public virtual bool IsChanged
{
    get
    {
        return _bIsChanged;
    }
    set
```

```
{
    _bIsChanged = value;
}
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
    if( this == obj ) return true;
    CategoryInfo castObj = null;
    try
    {
        castObj = (CategoryInfo)obj;
    }
    catch(Exception)
    {
        return false;
    }
    return ( castObj != null ) &&
        ( this._id == castObj.Id );
}
public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _id.GetHashCode();
    return hash;
}
#endregion
#region ICloneable methods
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion
}
#region Custom ICollection interface for CategoryInfo
public interface ICategoryInfoCollection : ICollection
{
    CategoryInfo this[int index]{get;
    set;
}
    void Add(CategoryInfo pCategoryInfo);
    void Clear();
}
[Serializable]
public class CategoryInfoCollection : ICategoryInfoCollection
{
    private IList<CategoryInfo> _arrayInternal;
    public CategoryInfoCollection()
    {
```

```
_arrayInternal = new List<CategoryInfo>();
}
public CategoryInfoCollection( IList<CategoryInfo> pSource )
{
    _arrayInternal = pSource;
    if(_arrayInternal == null)
    {
        _arrayInternal = new List<CategoryInfo>();
    }
}
public CategoryInfo this[int index]
{
    get
    {
        return _arrayInternal[index];
    }
    set
    {
        _arrayInternal[index] = value;
    }
}

public int Count { get { return _arrayInternal.Count; } }
}
public bool IsSynchronized { get { return false; } }
}
public object SyncRoot
{
    get
    {
        return _arrayInternal;
    }
}
public void CopyTo(Array array, int index)
{
    _arrayInternal.CopyTo((CategoryInfo[])array, index);
}
public IEnumerator GetEnumerator()
{
    return _arrayInternal.GetEnumerator();
}
public void Add(CategoryInfo pCategoryInfo)
{
    _arrayInternal.Add(pCategoryInfo);
}

public void Clear()
{

```

```
_arrayInternal.Clear();
}
public IList<CategoryInfo> GetList()
{
    return _arrayInternal;
}
}
#endregion

public class CategoryKind : ICloneable, ICategoryKind
{
    #region Member Variables
    protected string _id;
    protected string _kindname;
    protected string _kindcode;
    protected string _parentid;
    protected string _description;
    protected string _schoolid;
    #endregion
    #region Constructors
    public CategoryKind()
    {
    }
    public CategoryKind(string pId, string pKindName, string pKindcode, string
pParentid, string pDescription, string pSchoolId)
    {
        this._id = pId;
        this._kindname = pKindName;
        this._kindcode = pKindcode;
        this._parentid = pParentid;
    }
    public CategoryKind(string pId)
    {
        this._id = pId;
    }
    #endregion
    #region Public Properties
    public virtual string Id
    {
        get
        {
            return _id;
        }
    }
}
```